

Programowanie Obiektowe

Typy proste i referencyjne. Składowe obiektu

Dominik Kasprowicz

Instytut Mikroelektroniki i Optoelektroniki
Politechniki Warszawskiej

5 marca 2026

Spis treści

1 Java – typy proste i referencyjne

2 Java – składowe obiektu

Spis treści

1 Java – typy proste i referencyjne

2 Java – składowe obiektu

Java – typy danych

- **Typy proste** (ang. *primitives*):
 - znaki `char`,
 - liczby całkowite: `byte`, `short`, `int`, `long`,
 - liczby zmiennoprzecinkowe: `float`, `double`,
 - wartości logiczne `boolean`.
- **Typy referencyjne**:
 - tablice (nawet elementów typu prostego),
 - obiekty – instancje klas,
 - obiekty implementujące interfejsy.

Typy proste w języku Java – zakresy i wartości domyślne

Typ	Rozmiar (bity)	Minimum	Maksimum	Wart. dom.
byte	8	-2^7	$2^7 - 1$	0
short	16	-2^{15}	$2^{15} - 1$	0
int	32	-2^{31}	$2^{31} - 1$	0
long	64	-2^{63}	$2^{63} - 1$	0L
float	32	-2^{128}	$(2 - 2^{-23}) \cdot 2^{127}$	0.0f
double	64	-2^{1024}	$(2 - 2^{-52}) \cdot 2^{1023}$	0.0d
char	16	0	$2^{16} - 1$	'\u0000'
boolean	1	false	true	false

Typy proste w różnych językach

- **C i C++** – wymagają deklaracji typu i inicjalizacji:
 - `int n = 4;` OK,
 - `x = 4;` błąd kompilacji, jeśli `x` wcześniej nie zadeklarowana,
 - `int m;`
`printf("%d", m);` efekt nieprzewidywalny wg standardu...
 - ...choć większość kompilatorów inicjalizuje domyślnie zerem.
- **Python** – brak deklaracji typu:
 - `n = 4` typ dedukowany – `int`
 - `x = 4.0` typ dedukowany – `float`
- **Java** – konieczna deklaracja, wskazana inicjalizacja.
 - W niektórych kontekstach zmienna zostanie zainicjalizowana wartością domyślną (p. dalsza część wykładu).

Java – obiektywne odpowiedniki typów prostych

- Każdy typ prosty w języku Java ma swój odpowiednik obiektyw (ang. *wrapper type* – klasa **osłona**/**opakowująca**):
 - np. `int` → `Integer`.
- Typy obiektywne są wymagane w pewnych zastosowaniach:
 - np. w kolekcjach: listach, zbiorach itp.
`List<Integer> lista1; // OK`
`List<int> lista2; // BŁĄD!`
- Konsekwencje dla zajętości pamięci:
 - typ prosty: przechowywana jedynie wartość,
 - typ obiektyw: wartość oraz referencja do niej.

Java – obiektowe odpowiedniki typów prostych (pełna lista)

Typ prosty	Odpowiednik obiektowy
byte	Byte
short	Short
float	Float
int	Integer
long	Long
char	Character
double	Double
boolean	Boolean

Java – *autoboxing* i *unboxing*

- Konwersja między typem prostym i jego obiekowym odpowiednikiem:
 - często automatyczna i „przezroczysta” dla programisty...
 - ale *niezupełnie* bezkosztowa.
- **Autoboxing** („pakowanie”) – automatyczne tworzenie obiektu na podstawie odpowiadającego mu typu prostego.

```
LinkedList<Double> lista = new LinkedList<Double>();  
lista.add(3.14);
```

- **Unboxing** („rozpakowywanie”) – automatyczne tworzenie typu prostego na podstawie odpowiadającego mu obiektu.

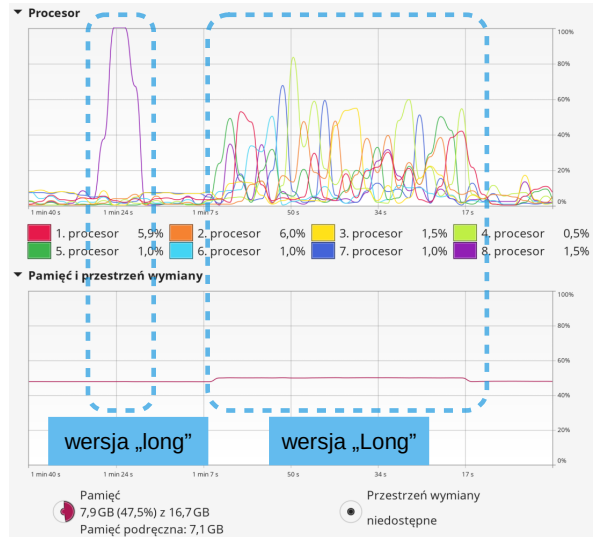
```
Float i = new Float(2.71);  
float j = i;
```

Autoboxing i unboxing – formalnie

- *Autoboxing* jest dokonywany przez kompilator, gdy typ prosty jest:
 - użyty jako argument metody, która wymaga odpowiadającego mu typu obiektowego lub
 - przypisany do zmiennej o odpowiadającym mu typie obiektowym.
- *Unboxing* jest dokonywany przez kompilator, gdy typ obiektowy jest:
 - użyty jako argument metody, która wymaga odpowiadającego mu typu prostego lub
 - przypisany do zmiennej o odpowiadającym mu typie prostym.
- Obiekty klas osłonowych są **niemutowalne** (podobnie jak np. **String**)
- **Częsty *autoboxing* i *unboxing* jest kosztowny**, ponieważ wymaga:
 - tworzenia obiektów (często jedynie tymczasowych),
 - ich niszczenia i czyszczenia pamięci.

Autoboxing i unboxing – koszt

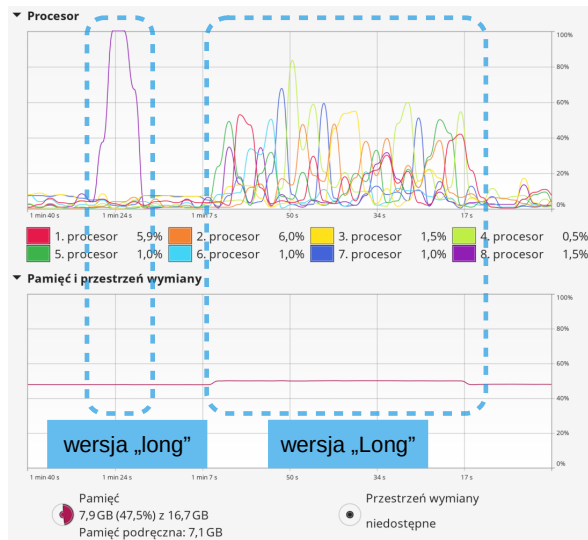
```
public class AutoboxingBezSensu {  
    public static void main(String[] args) {  
        final long LIMIT = 10L * Integer.MAX_VALUE;  
        long suma = 0L; // Dobra wersja  
        //Long suma = 0L; // Zła wersja  
        for (long i = 0; i < LIMIT; ++i)  
            suma += i;  
        System.out.println("Suma:" + suma);  
    }  
}
```



Autoboxing i unboxing – koszt

```
public class AutoboxingBezSensu {  
    public static void main(String[] args) {  
        final long LIMIT = 10L * Integer.MAX_VALUE;  
        long suma = 0L; // Dobra wersja  
        //Long suma = 0L; // Zła wersja  
        for (long i = 0; i < LIMIT; ++i)  
            suma += i;  
        System.out.println("Suma:" + suma);  
    }  
}
```

Wyjątek: **GraalVM** potrafi eliminować niepotrzebny *autoboxing* i *unboxing* – obie wersje programu wykonują się jednakowo szybko (niezależnie od kompilatora).



Java – konwersja typów

- Między typami prostymi, ich odpowiednikami obiektowymi i łańcuchami możliwa jest jawna konwersja.
- Możliwe także konwersje np. `double` $\longleftrightarrow$ `Integer`
- Na następnym slajdzie przykłady dla typów `int`, `Integer` i `String`.

Java – konwersja typów cd

- **int** → **Integer**:
 - `Integer(int i)` – konstruktor
 - `static Integer Integer.valueOf(int i)` – szybsza, przechowuje wyniki (-128, 127)
- **Integer** → **int**:
 - `int Integer.intValue()`
- **int** → **String**:
 - `static String String.valueOf(int i)`
- **String** → **int**:
 - `static int Integer.parseInt(String s, int radix)` – można wybrać podstawę
- **Integer** → **String**:
 - `static String String.valueOf(Integer i)`
 - `String Integer.toString()` – każdy typ obiektowy ma metodę `toString`
- **String** → **Integer**:
 - `Integer(String s)` – konstruktor
 - `static Integer Integer.valueOf(String s, int radix)`

Java – tablice

- Tablica jest typem referencyjnym
 - nawet tablica typów prostych, np. `int[]`
- Tablice wielowymiarowe mogą zawierać podtablice, także o różnych długościach:

```
String[][] tablica2d = {  
    {"krótki", "wiersz"},  
    {"to", "niezwykle", "wręcz", "długi", "wiersz"}  
};
```

- Rozmiaru tablicy nie można modyfikować po jej utworzeniu.
- Elementów tablicy nie można zabezpieczyć przed modyfikacją.
- Zamiast tablic preferowane są **kolekcje**: obiekty klas `Vector`, `Set`, `List` itp.
 - pozwalają na modyfikację struktury: dodawanie i usuwanie elementów,
 - mają „bardziej obiektowy” interfejs (udostępniają metody, iteratory itp.).

Klasa `java.util.Arrays`

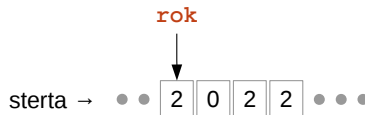
Klasa `java.util.Arrays` zawiera wiele **metod statycznych** operujących na tablicach, m.in.:

- `fill()` – wypełnia tablicę identycznymi elementami o podanej wartości.
- `sort()` – sortuje tablicę za pomocą udoskonalonej wersji algorytmu *quicksort*.
- `binarySearch()` – zwraca położenie (indeks) danej wartości w *posortowanej* tablicy, wyznaczone metodą **połowienia (bisekcji)**.
- `copyOf()` – zwraca kopię tablicy.
- `copyOfRange()` – zwraca kopię podanego zakresu elementów tablicy.
- `deepEquals()` – hierarchicznie sprawdza identyczność wielowymiarowych tablic, także zawierających elementy typu obiektowego.

Przypisywanie referencji – uwaga!

❶ Utworzenie tablicy:

```
int[] rok;  
rok = new int[]{ 2, 0, 2, 2 };
```



❷ Utworzenie kolejnej tablicy i przypisanie do niej *istniejącej* referencji:

```
rok = new int[]{ 1, 4, 1, 0 };
```



Przypisywanie referencji – uwaga cd

- Do „starych” danych brak już referencji – brak sposobu, by się do nich odwołać.
- A co z pamięcią zajmowaną przez „stare” dane?
- **Java** – pamięć zostanie zwolniona przy najbliższym uruchomieniu *Garbage Collectora*.
 - Kosztowna operacja!
- **Python** – pamięć zostanie zwolniona niemal natychmiast.
- **C++** – pamięć pozostanie niedostępna do końca działania aplikacji – **wyciek pamięci**.
 - A jeśli tą aplikacją jest usługa systemowa?

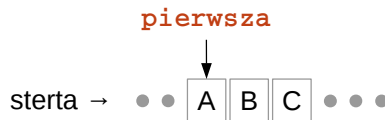
Problem ten dotyczy *wszystkich* typów referencyjnych: tablic i obiektów.

Kopiowanie tablicy – uwaga!

Jak wykonać kopię tablicy?

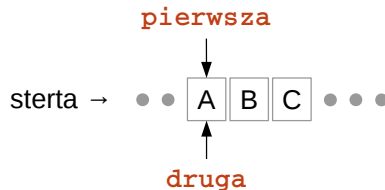
- 1 Utworzenie pierwszej tablicy:

```
char[] pierwsza = { 'A', 'B', 'C' };
```



- 2 Próba kopiowania (**źle!**):

```
char[] druga = pierwsza;
```



Kopiowanie tablicy cd.

- Skopiowana została *jedynie referencja* – adres danych na stercie.
- Referencja **druga** wskazuje na te same dane, co **pierwsza**.
- Wszystkie operacje wykonywane przez referencję **pierwsza** lub **druga** są wykonywane na tych samych danych.
- Jak uzyskać rzeczywistą kopię danych?

- Sposób 1:

```
char[] druga = new char[pierwsza.length];  
for (int i = 0; i < druga.length; ++i)  
    druga[i] = pierwsza[i];
```

- Sposób 2:

```
char[] druga = new char[pierwsza.length];  
System.arraycopy(pierwsza, 0, druga, 0, pierwsza.length);
```

Java – klasa String (łańcuch)

- Obiekt klasy **String** – ciąg (łańcuch) znaków UTF-16
- Obiekty **String** są **niezmienne** (ang. *immutable*):
 - nie można zmieniać ich zawartości „znak po znaku”, ale...
 - istniejącą referencję można przypisać do innego obiektu.
- Przykład:

```
String napis = "Kawa"; // utworzenie obiektu na podst. ciągu znaków
napis = "Java";        // przypisanie nowego obiektu String - OK
System.out.println(napis.charAt(1)); // odczytanie znaku 'a'
napis.charAt(2) = 'p'; // próba przypisania znaku - BŁĄD
```

Java – klasa String cd.

- Dla obiektów **String** zdefiniowany jest operator **konkatenacji** (łączenia) **+**
`napis = "Kawa" + " z " + "cukrem.";`
- Zagadka: ile w *sumie* znaków będą liczyły łańcuchy stworzone przez ten program?

```
String s = "";  
for (int i = 0; i < 10; ++i)  
    s = s + String.valueOf(i);
```

Java – klasa String cd.

- Dla obiektów **String** zdefiniowany jest operator **konkatenacji** (łączenia) **+**
`napis = "Kawa" + " z " + "cukrem.";`
- Zagadka: ile w *sumie* znaków będą liczyły łańcuchy stworzone przez ten program?

```
String s = "";  
for (int i = 0; i < 10; ++i)  
    s = s + String.valueOf(i);
```

```
0  
01  
012  
0123  
01234  
012345  
0123456  
01234567  
012345678  
0123456789
```

W razie konieczności częstych modyfikacji lepiej użyć **StringBuilder** lub **StringBuffer**.

Java – klasa String cd.

- Każdy obiekt Javy ma reprezentację tekstową, zwracaną przez metodę `toString`.
 - Dla własnych klas można tę metodę dowolnie zdefiniować...
 - ...lub użyć implementacji domyślnej, zwracającej:
`NazwaKlasy@unikatowy_identyfikator_obiektu`
- Umożliwia to użycie obiektu *dowolnej* klasy wszędzie tam, gdzie wymagany jest `String`.
- Argumenty wywołania programu są przekazywane do funkcji `main` jako tablica obiektów klasy `String`:

```
public static void main(String[] args) {  
    for (String s : args)  
        System.out.println(s);  
}
```

Klasa `java.lang.String` – użyteczne metody

- `length` – zwraca długość łańcucha, tj. liczbę znaków Unicode.
- `isEmpty` – zwraca `true`, jeśli długość łańcucha wynosi zero.
- `charAt` – zwraca znak (`char`) na podanej pozycji (liczonej od zera).
- `codePointAt` – jak wyżej, ale zwraca liczbę interpretowaną jako symbol Unicode.
- `compareTo` – porównuje *leksykograficznie* dany `String` z innym obiektem tego typu.
- `indexOf` – zwraca pozycję pierwszego wystąpienia podanego znaku lub łańcucha.
- `valueOf` – zwraca reprezentację argumentu (np. typu prostego) jako `String` (metoda statyczna).

Klasa `java.lang.String` – użyteczne metody cd.

- `concat` – zwraca `String` powstały po połączeniu dwóch łańcuchów.
- `join` – jak wyżej, ale pierwszy argument trafia pomiędzy każdą parę łączonych łańcuchów.
- `replace`, `replaceAll` – zwraca *kopię* łańcucha, w której wyrażenie pasujące do podanego wzorca zostało zastąpione podanym łańcuchem.
- `split` – używa podanego separatora do rozbicia łańcucha, wynik jest tablicą łańcuchów.
- `strip`, `stripLeading`, `stripTrailing` – usuwa **białe znaki**: spacje, tabulacje, znaki końca wiersza itp. z obu stron/początku/końca łańcucha.
- `toLowerCase`, `toUpperCase` – zwraca *kopię* łańcucha zapisaną małymi/wielkimi literami.

Java – klasy `StringBuilder` i `StringBuffer`

- Ciągi znaków, które można modyfikować bez konieczności tworzenia kopii.
- Interfejs obu klas jest niemal identyczny.
- `StringBuilder` – operacje znacznie szybsze, ale...
- `StringBuffer` – bezpieczny dostęp z wielu wątków.

```
StringBuilder tekst = new StringBuilder("Java jest trudna");  
tekst.insert(10, "nie");  
System.out.println(tekst);  
tekst.replace(10, 19, "łatwa");  
System.out.println(tekst);  
tekst.append(" i intuicyjna");
```

Spis treści

1 Java – typy proste i referencyjne

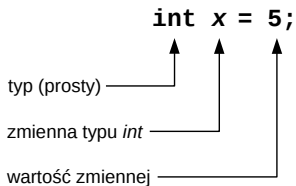
2 Java – składowe obiektu

Java – hierarchia klas

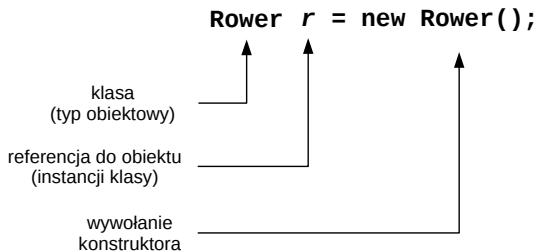
- Wszystkie klasy dziedziczą po `java.lang.Object`
 - nawet klasy zdefiniowane przez użytkownika oraz tablice!
- Metody zdefiniowane w klasie `Object`:
 - `boolean equals(Object obj)` zwraca true, jeśli `obj` jest referencją do danego obiektu,
 - `Object clone()` zwraca kopię danego obiektu,
 - `Class<?> getClass()` zwraca klasę, której instancją jest dany obiekt,
 - `int hashCode()` zwraca liczbę reprezentującą dany obiekt (z reguły unikatową, choć nie ma takiej gwarancji),
 - `String toString()` zwraca ciąg znaków reprezentujący dany obiekt,
 - ...oraz metody pozwalające aplikacjom wielowątkowym na bezpieczną pracę z danym obiektem.

Typy proste i obiektowe – nazewnictwo

Zmienna typu prostego

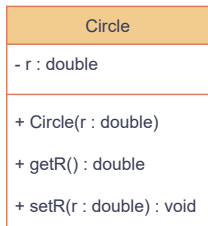


Zmienna typu obiektowego



Typy obiektowe – nazewnictwo

Diagram klas UML



Kod Java

```
public class Circle {  
    private double r;  
    public Circle(double r) {  
        this.r = r;  
    }  
    public double getR() {  
        return r;  
    }  
    public void setR(double r) {  
        this.r = r;  
    }  
}
```

Diagram illustrating the mapping between the Java code and the UML class diagram components:

- `public class Circle {` → nazwa klasy
- `private double r;` → pole (atrybut)
- `public Circle(double r) {` → konstruktor
- `public double getR() {` → metoda dostępowa („getter”)
- `public void setR(double r) {` → metoda dostępowa („setter”)

Groupings:

- `private double r;` and the three methods are grouped by a large bracket on the right labeled **składowe**.
- The three methods are grouped by a bracket on the right labeled **metody**.

Uwagi językowe: obiekt a klasa, użytkownik klasy

- Pamiętajmy:
 - **Klasa** jest typem (obiektowym), np. **Pies**.
 - **Obiekt** jest konkretną instancją danej klasy, np. **mójTofik**.
- W literaturze słowo „klasa” bywa *błędnie* używane w znaczeniu „obiekt tej klasy”, np.
 - „Funkcje które używają wskaźników lub referencji do klas bazowych [...]” (*Zasada podstawienia Liskov*, polska Wikipedia, dostęp 24.03.2025).
- **Użytkownik klasy** oznacza *programistę* używającego klasy w swoim projekcie.
 - Z reguły mamy na myśli programistę *spoza* zespołu twórców klasy.
 - Użytkownik ma dostęp do kodu bajtowego klasy i jej dokumentacji, nie do kodu źródłowego.
 - Użytkownik może czasem zmodyfikować („uszkodzić”) *obiekt*, ale nie klasę (Java to nie Python).

Składowe obiektu

- Pola obiektu reprezentują stan tego konkretnego obiektu (konkretnej **instancji klasy**).
 - Odwołanie do pola obiektu: `obiekt.pole`
 - Przykład: `smallCircle.r`
- Metody obiektu reprezentują czynności, które *ten obiekt* może wykonać.
 - Wywołanie metody obiektu („na rzecz obiektu”): `obiekt.metoda(...)`
 - Przykład: `bigCircle.getR();` – zwraca promień konkretnego okręgu `bigCircle`.
- Składową klasy może być także inna klasa (**zagnieżdżona**).
 - Pełni rolę użytkową na rzecz swojej klasy zewnętrznej.
- Klasa w Javie też jest obiektem (klasy `Class`).

Składowe statyczne (składowe klasy)

- **Zmienne statyczne** reprezentują stan całej klasy obiektów, a nie konkretnego obiektu.
 - Typowy przykład zmiennej statycznej: licznik instancji obiektów danej klasy.
 - `private static int ileRowerówPowstało` – mogłaby zostać zdefiniowana w klasie `Rower`.
- **Stałe statyczne**: stałe charakterystyczne dla dziedziny reprezentowanej przez klasę, np.
 - `public static final double PI` – zdefiniowana w klasie `Math`.
 - `public static final int MAX_VALUE` – zdefiniowana w klasie `Integer` maksymalna wartość, jaką może przyjmować liczba typu `int`.
- **Metody statyczne** reprezentują zachowanie całej klasy obiektów, np.
 - `public static double abs(double a)` – zdefiniowana w klasie `Math`; zwraca wartość bezwzględną argumentu.

Składowe statyczne cd.

- Sposób użycia:
 - `Klasa.pole_statyczne`, np. `Math.PI`, `Integer.MAX_VALUE`
 - `Klasa.metoda_statyczna()`, np. `Integer.parseInt("2022", 10);`
- Składowe statyczne mogą być używane nawet, gdy nie istnieje żaden obiekt danej klasy.
 - Np. klasa `Math` zawiera *wyłącznie* metody statyczne i pola statyczne, nie ma za to publicznego konstruktora – niemożliwe jest utworzenie obiektu klasy `Math`.
- Metody statyczne nie są skojarzone z żadnym obiektem, zatem *nie mogą* używać:
 - składowych instancji (tj. składowych niestatycznych),
 - referencji `this`.

Java – metody obiektów

- **Metoda** to funkcja będąca składową obiektu (lub klasy – **metody statyczne**).
- W wielu językach obiektowych (np. **C++**, **Python**, **Swift**) oprócz metod można definiować tzw. **funkcje swobodne** (**wolne**, ang. *free functions*) tj. niebędące metodami żadnego obiektu ani klasy.
- **Java** – prawie każda funkcja musi być metodą jakiegoś obiektu lub klasy.
- Jedyne wyjątek: **main()** może być funkcją swobodną (od Javy 25).
 - Upraszcza to pisanie bardzo krótkich programów.
 - W „prawdziwych” aplikacjach **main** jest wciąż metodą statyczną.

Metody – parametry a argumenty

- **Parametry** wywołania funkcji są widoczne w jej definicji, np.

```
int dodaj(int x, int y) { return x+y; }
```

- W momencie wywołania metody w miejsce parametrów podstawiamy **argumenty**, tj. konkretne wartości:

```
int a = 2;
```

```
int wynik = dodaj(a, 3);
```

Java – przekazywanie do metod argumentów typów prostych

- Argumenty *typów prostych* są zawsze przekazywane do metody *przez wartość*.
- Wewnątrz ciała funkcji operacje są wykonywane na lokalnej kopii argumentu.
- Przypisanie wewnątrz metody nowej wartości do argumentu nie powoduje **skutków ubocznych** (nie jest widoczne poza ciałem funkcji).
- Przykład:

```
void zmniejsz(int x) { x = x-1; } // Metoda bez sensu!  
int a = 4;  
zmniejsz(a);  
// Dalej a = 4.
```

Java – przekazywanie do metod argumentów typów referencyjnych

Dla argumentów *typu referencyjnego* jest identycznie:

- metoda operuje na *lokalnej kopii* argumentu...
- ...ale tym argumentem jest *adres oryginalnego* obiektu lub tablicy!

```
import java.awt.Point;
//...
// Modyfikujemy pola obiektu Point o adresie 'p'.
void przenieśDo(Point p, int m, int n) { p.x = m; p.y = n; }
// Do zmiennej lokalnej (!) 'p' przypisujemy nowy adres.
void przypiszInnyPunkt(Point p, int m, int n) { // Metoda bez sensu!
    p = new Point(m, n);
}
```

```
Point pkt = new Point(10, 0);
przenieśDo(pkt, 5, 15);      // Teraz pkt ma współrzędne (5, 15).
przypiszInny(pkt, 100, 200); // pkt ma dalej współrzędne (5, 15).
```

Java – sygnatury metod

- Nazwa metody razem z listą parametrów tworzą **sygnaturę** metody.
- W klasie *nie można* zdefiniować dwóch metod o identycznej sygnaturze, różniących się jedynie typem zwracanym.
⇒ Byłyby nierozróżnialne w momencie wywołania!
- W klasie można natomiast zdefiniować wiele metod o tej samej nazwie, różniących się typem i/lub liczbą parametrów – **przeciążanie** metody (ang. *method overloading*).

```
void    drawPoint(int x,   int y)    { /*...*/ }  
void    drawPoint(float x, float y) { /*...*/ } // OK, ale łatwo pomylić z poprzednią.  
void    drawPoint(Point p) { /*...*/ }  
boolean drawPoint(Point p) { /*...*/ } // Konflikt nazw - błąd czasu kompilacji!
```

Sygnatury metod – *nieformalne* konwencje

- Nazwa metody powinna zaczynać się od czasownika.
- Konwencje nazewnictwa:
 - Metody odpowiadające na pytanie „czy”:
`boolean is...(), boolean has...()`
 - Metody nadające wartość („**setter**”):
`void set...(typ nowa_wartość)`
 - Metody odczytujące wartość („**getter**”):
`typ_zwracany get...()`
 - Metody konwertujące argument (lub `this`) do typu `X`:
`X asX(), X toX()`
- Lista parametrów metody nie powinna być dłuższa niż 4 elementy.
 - Warto tworzyć osobne klasy grupujące parametry:
`drawPoint(int x, int y) → drawPoint(Point p)`

Zmienne lokalne metod

- Wewnątrz metody można deklarować zmienne i stałe.
 - Widoczne wyłącznie w ciele tej metody.
- Nie są to pola lecz **zmienne lokalne**.
- *Nie jest* im nadawana domyślna wartość...
 - ...w odróżnieniu od pól, domyślnie inicjalizowanych „zerową” wartością.
 - Próba kompilacji bez ich inicjalizacji nie powiedzie się.

```
public class WartościDomyślne {  
    static int x;  
    public static void main(String[] args) {  
        int y;  
        System.out.println("x = " + x);  
        // System.out.println("y = " + y);  
    }  
}
```

Konstruktory

- Konstruktor jest specjalną metodą, wywoływaną podczas tworzenia obiektu.
- Konstruktor inicjuje pola tworzonego obiektu:

```
public Circle(double r) { this.r = r; }
```
- Klasy miewają kilka konstruktorów, różniących się liczbą i/lub rodzajem parametrów.
 - Rodzajem, bo obiekty pewnych klas można stworzyć na podstawie różnych typów:
 - `String(char[] value)` – tworzy łańcuch na podstawie tablicy znaków.
 - `String(byte[] bytes)` – tworzy łańcuch na podstawie tablicy bajtów.
 - Liczbą, bo pola niezainicjowane *explicite* można zainicjować wartościami domyślnymi:
 - `Dialog(Window owner, String title)` – tworzy okno dialogowe o podanej nazwie.
 - `Dialog(Window owner)` – tworzy okno dialogowe bez nazwy.

Konstruktory cd.

- Jak odróżnić konstruktor od innych metod?

```
public Circle(double r) { ... }
```

- Nazwa konstruktora – identyczna jak nazwa klasy (i też rozpoczyna się wielką literą).
 - Konstruktor niczego nie zwraca – nawet `void`.
 - Zwyczajowo w definicji klasy konstruktory występują przed innymi metodami.
- Konstruktor bezparametrowy jest nazywany **konstruktorem domyślnym**.
 - Jeśli twórca klasy nie zdefiniuje *żadnego* konstruktora, kompilator *wygeneruje* konstruktor domyślny. Taki konstruktor:
 - Nie będzie widoczny w kodzie (ale będzie widoczny dla kompilatora).
 - Zainicjuje wszystkie pola wartościami domyślnymi (dla typów prostych zerami, dla typów referencyjnych – `null`-ami).
 - Niemożność wywołania konstruktora $\implies$ niemożność utworzenia obiektu.

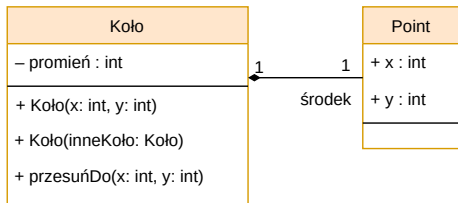
Konstruktor kopiujący

- **Konstruktor kopiujący** tworzy obiekt na podstawie innego obiektu danej klasy.

```
public Circle(Circle c) { this.r = c.r; // Ew. inne operacje }
```

- A jeśli pewne pola kopiowanego obiektu są typu referencyjnego (obiekty, tablice)?
 - Samo skopiowanie tych pól (tj. adresów do danych) nie wystarczy – referencje w kopii będą wskazywać na te same dane, co referencje w oryginale (**płytkie kopiowanie**).
 - Rozwiązanie: należy skopiować również wskazywane dane: dla obiektów – wywołując konstruktor kopiujący, dla tablic – np. poprzez **arraycopy** (**głębokie kopiowanie**).
 - Często pola typu referencyjnego mają pola typu referencyjnego itd. – proces hierarchiczny.
 - Jeżeli konstruktory kopiujące pól (i ich pól oraz pól ich pól itd.) są napisane poprawnie, powstaje kopia *w pełni niezależna* od oryginału.
- Jeśli brak konstruktora kopiującego, *nie zostanie* on wygenerowany automatycznie!

Konstruktor kopiujący – zła implementacja (płytką kopia)



```
class Koło {
    private int    promień;
    private Point  środek;

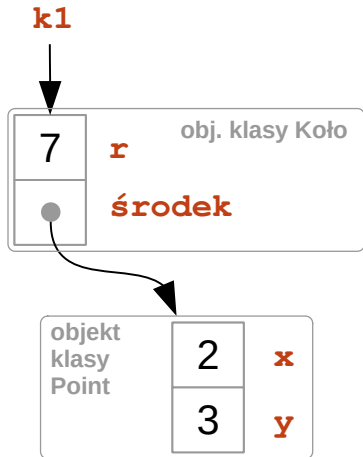
    // Konstruktor.
    public Koło(int r, int x, int y) {
        promień = r;
        środek  = new Point(x, y);
    }

    // Kopiowanie płytkie w konstr. kopiującym.
    // TAK NIE ROBIMY!
    public Koło(Koło inneKoło) {
        promień = inneKoło.promień;
        środek  = inneKoło.środek;
    }

    // Tu metoda przesunDo()
}
```

Konstruktor kopiujący – zła implementacja (płytką kopia) cd.

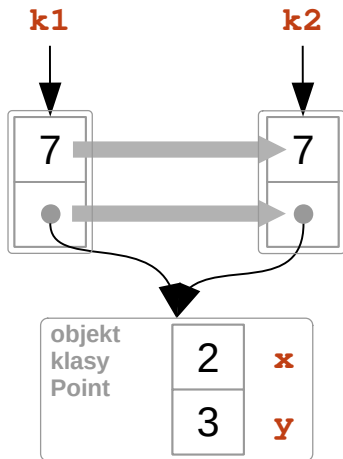
```
Koło k1 = new Koło(7,2,3);
```



```
class Koło {  
    private int    promień;  
    private Point  środek;  
  
    // Konstruktor.  
    public Koło(int r, int x, int y) {  
        promień = r;  
        środek  = new Point(x, y);  
    }  
  
    // Kopiowanie płytkie w konstr. kopiującym.  
    // TAK NIE ROBIMY!  
    public Koło(Koło inneKoło) {  
        promień = inneKoło.promień;  
        środek  = inneKoło.środek;  
    }  
  
    // Tu metoda przesunDo()  
}
```

Konstruktor kopiujący – zła implementacja (płytką kopia) cd.

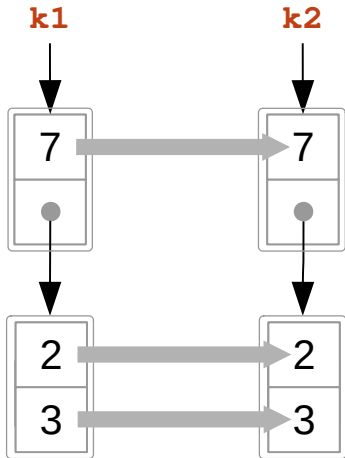
```
Koło k1 = new Koło(7,2,3);  
Koło k2 = new Koło(k1);
```



```
class Koło {  
    private int    promień;  
    private Point  środek;  
  
    // Konstruktor.  
    public Koło(int r, int x, int y) {  
        promień = r;  
        środek  = new Point(x, y);  
    }  
  
    // Kopiowanie płytkie w konstr. kopiującym.  
    // TAK NIE ROBIMY!  
    public Koło(Koło inneKoło) {  
        promień = inneKoło.promień;  
        środek  = inneKoło.środek;  
    }  
  
    // Tu metoda przesunDo()  
}
```

Konstruktor kopiujący – dobra implementacja (kopia głęboka)

```
Koło k1 = new Koło(7,2,3);  
Koło k2 = new Koło(k1);
```



```
public Koło(Koło inneKoło) {  
    promień = inneKoło.promień;
```

```
    // Źle - kopiujemy wyłącznie referencję  
    // do "środka".
```



```
    // środek = inneKoło.środek;
```

```
    // Dobrze - tworzymy nowy obiekt  
    // poprzez wywołanie jego konstruktora  
    // kopiującego (zakładamy, że  
    // jest poprawnie napisany).
```



```
    środek = new Point(inneKoło.środek);  
}
```

Tworzenie obiektu

Jak utworzyć instancję obiektu o wielu polach?

- 1 Zawsze używać konstruktora inicjującego wszystkie pola.
 - Niewygodne.
- 2 Użyć konstruktora bezparametrowego i nadawać wartości polom *po utworzeniu* obiektu.
 - Rozwlekły kod.
 - Dla pól stałych (**final**) – niewykonalne.
- 3 Stworzyć wiele konstruktorów z różnymi listami parametrów.
- 4 Zgrupować parametry konstruktora w bardziej złożone obiekty.
- 5 Użyć wzorca Budowniczy (kolejny wykład).

Klasa z wieloma konstruktorami – przykład

```
public class Zespolona {  
    private double re = 0.0;  
    private double im = 0.0;  
    public Zespolona() { }  
    public Zespolona(double x) { re = x; }  
    public Zespolona(double x, double y) { re = x; im = y; }  
    //...
```

```
Zespolona z1 = new Zespolona();  
Zespolona z2 = new Zespolona(1.0);  
Zespolona z3 = new Zespolona(1.0, 2.0);
```

Klasa z wieloma konstruktorami – domyślne wartości pól stałych

- **Problem** – polom stałym (**final**) można nadać wartość tylko raz
 - jeśli zrobimy to w momencie deklaracji, to w konstruktorze będzie to już niemożliwe:
error: cannot assign a value to final variable.
- Rozwiązanie: w każdym konstruktorze inicjować wszystkie pola
 - dla każdego brakującego parametru użyć wartości domyślnej.

```
public class Zespolona2 {  
    private final double re;  
    private final double im;  
    public Zespolona2() { re = 0.0; im = 0.0; }  
    public Zespolona2(double x) { re = x; im = 0.0; }  
    public Zespolona2(double x, double y) { re = x; im = y; }  
    //...
```

Konstruktor teleskopowy – słowo kluczowe **this**

Uproszczenie kodu z poprzedniego slajdu – **konstruktor teleskopowy**

```
public class Zespolona3 {  
    private final double re;  
    private final double im;  
  
    public Zespolona3(double x, double y) { re = x; im = y; }  
    public Zespolona3(double x) { this(x, 0); }  
    public Zespolona3() { this(0); }  
    //...
```

„Grupowanie” argumentów metod – przykład

- Klasa `java.awt.Rectangle`:

```
public class Rectangle {  
    private int x; // lewy-górny róg  
    private int y; // lewy-górny róg  
    private int height;  
    private int width;  
  
    Rectangle(int x, int y, int width, int height) {...}  
    Rectangle(Point p, Dimension d) {...}  
    //...
```

- Tę technikę warto stosować także do innych metod, niebędących konstruktorami.
- Lista parametrów metody nie powinna być dłuższa niż **4** elementy (nieformalna zasada).
- Więcej o konstruktorach przy okazji omawiania dziedziczenia.

Tworzenie obiektu – wzorzec Budowniczy (ang. *Builder*)

```
public class Sprinter {  
    private final String name;  
    private final double pb100m;  
    private final double pb200m;  
    private final double pb400m;  
  
    private Sprinter(String n, double pb100, double pb200, double pb400) {  
        name    = n;  
        pb100m  = pb100;  
        pb200m  = pb200;  
        pb400m  = pb400;  
    }  
  
    static class Builder {...}  
    //...
```

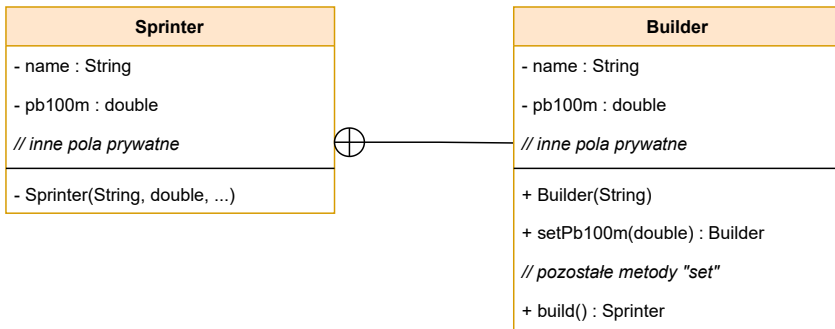
Tworzenie obiektu – wzorzec Budowniczy cd.

```
static class Builder {  
    private final String name; // Par. wymagany!  
    // Wartości domyślne  
    private double pb100m = -1.0;  
    private double pb200m = -1.0;  
    private double pb400m = -1.0;  
  
    // Konstruktor - nazwisko jest wymagane!  
    public Builder(String n) { name = n; }  
  
    public Builder setPb100m(double x) {  
        pb100m = x;  
        return this;  
    } // Analogicznie dla pozostałych pól:  
    public Builder setPb200m(double x) {...}  
    public Builder setPb400m(double x) {...}  
    public Sprinter build() {  
        return new Sprinter(name, pb100m, pb200m, pb400m);  
    }  
}
```

```
Sprinter usain = new Sprinter.Builder("U. Bolt")  
    .setPb400m(45.28)  
    .setPb100m(9.58)  
    .setPb200m(19.19)  
    .build();
```

```
Sprinter marian = new Sprinter.Builder("M. Woronin")  
    .setPb100m(10.00)  
    .build();
```

Klasa zagnieżdżona – notacja UML



Rekordy (**record**) – alternatywa dla trywialnych klas

```
class Owoc {  
    private final String rodzaj;  
    private final double cenaKg;  
  
    public Owoc(String rodzaj, double cenaKg) {  
        this.rodzaj = rodzaj;  
        this.cenaKg = cenaKg;  
    }  
  
    public String rodzaj() { return this.rodzaj; }  
    public double cenaKg() { return this.cenaKg; }  
    public String toString() {  
        return "Owoc[" +  
            "rodzaj=" + this.rodzaj + "," +  
            "cenaKg=" + this.cenaKg + "];"  
    }  
    // Tu sensowne implementacje metod  
    // equals i hashCode.  
};
```

Rekordy (**record**) – alternatywa dla trywialnych klas

```
class Owoc {  
    private final String rodzaj;  
    private final double cenaKg;  
  
    public Owoc(String rodzaj, double cenaKg) {  
        this.rodzaj = rodzaj;  
        this.cenaKg = cenaKg;  
    }  
  
    public String rodzaj() { return this.rodzaj; }  
    public double cenaKg() { return this.cenaKg; }  
    public String toString() {  
        return "Owoc[" +  
            "rodzaj=" + this.rodzaj + "," +  
            "cenaKg=" + this.cenaKg + "];"  
    }  
  
    // Tu sensowne implementacje metod  
    // equals i hashCode.  
};
```

Identyczna funkcjonalność:

```
record Owoc(String rodzaj, double cenaKg) { };
```